J. Inst. Eng. India Ser. B (August 2019) 100(4):309–319
https://doi.org/10.1007/s40031-019-00413-z

ORIGINAL CONTRIBUTION

An Online Testing Scheme for Detection of Gate Faults in ESOP-Based Reversible Circuit

Bappaditya Mondal[1] · Chandan Bandyopadhyay[1] · Anirban Bhattacharjee[1] · Hafizur Rahaman[1]

Received: 7 August 2018 / Accepted: 4 June 2019 / Published online: 11 June 2019
© The Institution of Engineers (India) 2019

Abstract In the present era of computation, reversible computing has emerged as promising computing paradigm before the research community. Consequently, efficient design of reversible circuit has received immense attention in design industry. But not only the cost efficient designs get the priority but developing methodologies for testing of such circuit bear high value. Emphasizing on this need, in this work, we have introduced an online testing scheme for indentifying two major gate faults—missing-gate fault and appearance fault in ESOP-based reversible designs. In the testing process, initially, the selected input function is transformed to its corresponding testable circuit in which additional gates and lines (ancillary lines) are appended in the primary design. Next, we generate fault-specific test vectors which later are executed over the circuit for identifying the possible faults present in the design. This approach has been successfully tested on a wide spectrum of benchmark specifications to ascertain its functional correctness, and the computed results are tabulated at the end of the article. Comparison with existing testing algorithms is given in the result tables, and a substantial improvement over reported work's results is projected in those result sets.

✉ Chandan Bandyopadhyay
chandanb.iiest@gmail.com

Bappaditya Mondal
bappa.arya@gmail.com

Anirban Bhattacharjee
anirbanbhattacharjee330@gmail.com

Hafizur Rahaman
hafizur@it.iiests.ac.in

[1] Department of Information Technology, Indian Institute of Engineering Science and Technology, Shibpur 711103, India

Keywords Exclusive-or sum-of-product (*ESOP*) · Appearance fault (AF) · Missing-gate faults (*MGF*)

Introduction

With the advancement of technological footprints, miniaturization in IC industry is in continuous trend. But contrary, the high packing density of transistors in ICs has caused a serious concern as high energy dissipation [1] from such highly packed transistors damage the circuits in ICs. So, alternative technology is required to contain this effect. Though the term "*reversible circuit*" is not a newer one and in past years, various claims [2] on the energy efficient property of this special type of circuit have been made, but the recent experimental verification by Berut et al. [3] has boosted the confidence of designers to further investigate in this field. On the other side as the quantum circuit [4] also follows the principle of reversibility, the implementation of quantum functionality using reversible circuit is possible. Reversible circuit not only has the dominance in the field of quantum circuit design, but it too has applications in adiabatic computing [5, 6], cryptography and optical computing.

In the last couple of years, several progresses have been made on efficient design strategies of reversible circuit. Synthesis algorithms have been developed for making the designs of reversible circuit generic. But, not only designing the cost efficient circuits get the high importance but simultaneously developing testing algorithms [7–14] for checking the correctness of such designs find popularity. In recent time, several promising works on efficient testing strategies are reported, where improved algorithms are formulated to make the testing process easier. Prior

detailing on existing testing works, in the following we summarize the preliminaries on reversible logic and fault models for making the readers familiar with associated terms and notations.

Preliminaries

Definition 1 A circuit C_{rev} having circuit depth of h acting over a group of circuit lines $L = \{c_1, c_2, \ldots, c_n\}$ is considered reversible provided satisfying the following three conditions.

(i) the number of inputs (m) is identical with the number of outputs (n)
(ii) if the circuit is fan-out free
(iii) circuit consists with reversible gates only

Like as Boolean logic gates used to design classical circuits, there are well-known reversible gates like Toffoli [15], Fredkin [16], Feynman [17] used to design reversible circuits. A reversible gate can be defined as follows.

Definition 2 A reversible gate G can be defined as $G_{rev}(C; T)$, where parameter C, T denotes control line set and target line set, respectively. In that gate G_{rev}, set C may contain an empty value but T must have at least one target line in such a way that $C \cap T = \Phi$.

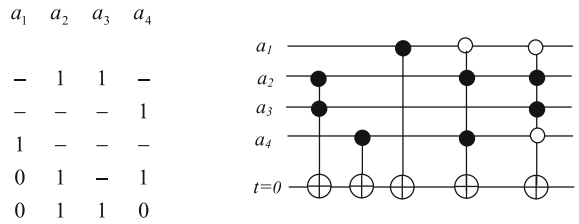
A reversible circuit may have different designs and such variations in design depend on the type of algorithm deployed or heuristic employed. Among the several design models, due to the scalable feature property, ESOP (Exclusive Sum-Of-Products) [18]-based representation has been found as one of the widely used design model for reversible circuit.

- *ESOP-based design:*

ESOP is one of the representation classes under *Sum-Of-Products* (*SOP*), in which the product terms in an expression are separated by the XOR ($\oplus$) operator. To represent an n-input m-output function in ESOP configuration, it requires $(n + m)$ circuit lines whereby the first n lines act as control inputs and the rest m lines are functional outputs.

Cube list a special data structure from which the ESOP designs are formed. Such cube list contains the detail gate specification and control structure for the ESOP representation. Each cube in the cube list denotes a gate in the design. For an ease of understanding, an ESOP cube list along with its equivalent ESOP representation is given in Fig. 1a, b, where it can be seen that each of the cube from the list has been mapped to an equivalent gate and finally a complete design is formed Fig. 1b.

Though ESOP-based designs have very simplified structures but such designs may contain fault due to disappearance of gates or control nodes. Here, we are discussing about such fault models and their properties.



(a) Cube list for $f(a_1, a_2, a_3, a_4) = a_2a_3 \oplus a_4 \oplus a_1 \oplus \bar{a}_1a_2a_4 \oplus \bar{a}_1a_2a_3\bar{a}_4$. **(b)** Equivalent ESOP representation

Fig. 1 **a** Cube list for $f(a_1, a_2, a_3, a_4) = a_2a_3 \oplus a_4 \oplus a_1 \oplus \bar{a}_1a_2a_4 \oplus \bar{a}_1 a_2a_3\bar{a}_4$, **b** equivalent ESOP representation

- *Different fault models and their properties*

Faults [19–21] in circuits may originate due to several reasons. It may be due to missing of control nodes, missing of entire gate or may be due to appearance of new gates in the given circuit. Based on the type of errors, faults are mainly categorized into four classes—single missing-gate fault (*SMGF*), repeated gate fault (*RGF*), cross-point fault (*CPF*) and multiple missing-gate fault (*MMGF*).

Definition 3 If a gate completely disappears from a circuit by changing its functionality, then the said fault is termed as single missing-gate fault.

Definition 4 A fault can be termed as repeated gate fault if the same gate consecutively reappears in the design which causes a functional change in the circuit.

Definition 5 If the existing control nodes of a gate either disappears or a new control appears for the same gate then the fault is known as cross-point fault.

Furthermore, cross-point fault [22] can be categorized into the following two sub-classes depending on the fault position.

- *Disappearance fault: Missing of at least one control node from a gate causes this fault.*
- *Appearance fault: It occurs when a new control node unwontedly appears or wrongly placed in a gate.*

Definition 6 Disappearance of two or more consecutive gates from a circuit causes such type of fault.

The fault models that we have discussed till so far have been studied in various research articles, where the authors mainly have tried to address the faults by developing appropriate fault detection mechanism. Now, here we are discussing some of those works and highlighting their contributions.

Related Works and Contributions

Detection of stuck-at-fault by finding an adaptive tree-based solution is proposed in [7], where initially the fault in half of the circuit is detected and then by exploiting the reversible property, mirror image of the first part is created in remaining part of the circuit and then the fault is detected. This scheme has shown very optimistic results and generates fast solutions.

Furthermore, in way to generalize the "stuck-at" fault detection for all k-CNOT based circuit, a universal testing scheme is developed in [8], where along with the stuck-at fault detection technique authors have presented the way for modifying an n-wire reversible circuit to a universal test set of size 3.

Taking one step more, a novel approach for fault identification is developed in [9], where an $(n \times n)$ reversible netlist is considered with k-CNOT gates for tracking the possible faults in the design. In way to make the detection simpler, in the work, copy gates and an additional line are added in the testable design. Though this addition created some design overhead, but the modified testable design is considered suitable for examining all SMGFs, RGFs and PMGFs in the given circuit.

Instead of using the traditional test sets in detecting faults, a faster mechanism is developed in [10], where Boolean generator-based testing technique has been introduced.

Application of Reed–Muller form in detection and localization of all SMGFs in ESOP-based designs has been presented in [11], where the positive polarity Reed–Muller expression of a circuit is obtained first and then the faulty circuit expression is XOR-ed with the PPRM expression to find the fault in the design.

To make the faults detection easier, not only fault-specific test vectors have been generated but the way of making simpler testable designs also have been explored and such a work is reported in [12], where extra inputs and additional k-CNOT gates are added in the design to make the design testing friendly. This modified design considers a universal test set of size $(n + 2)$ and thereby covers all the existing fault models.

In way to reduce the complexity of test vector generation process, an improved approach for testing SMGFs, RGFs and MMGFs in a reversible netlist with minimum number of test vectors in presented in [13].

The testing schemes that we have stated till so far belong to offline testing category as the schemes are not able to find faults while circuits are in operational mode.

To address this drawback, the concept of online testing mechanism introduced and here now we are discussing some of the works from online testing.

Online testing of all $(n \times n)$ and d depth reversible circuits using $2d$ garbage outputs along with $(2d + 1)$ ancille lines has been shown in [14]. Performing fast online testing using modified testable design is shown [23], where three new (4×4) gates have been added in the circuit to make the design simpler.

An online-based testing technique for reversible netlist is shown in [24], where three additional gates are appended in the testable netlist for making the testing process faster.

Efficient way of performing online testing using improved testable design has been introduced in [25–27], where the incorporation of checker circuit and dual rail coding technique is followed to make the fault detection faster.

As the testable designs, sometime contain additional lines but this addition creates design overhead in the testing process. To contain this effect, an online testing scheme without using any ancille has been introduced in [28], where all SMGFs have been successfully detected without making any overhead in the design.

Still so far we have seen various testing schemes and their approaches but design of efficient testing algorithms always remain a priority. In our work, we have developed such a scheme which not only makes fault detection faster but also can test very large circuits in online mode. Our developed approach can detect MGFs and appearance faults in ESOP circuit very easily without creating much overhead in the design. We have tested our technique over a wide spectrum of benchmarks and improvements in performance metrics over existing works have been observed.

In the next section, we state our developed testing scheme. Experimental results and comparison with existing related works are summarized in the next section—"Experimental Results" section. Finally, the work concludes in "Conclusion" section.

Proposed Technique

In previous section, we have seen that *ESOP* is a widely recognized circuit design technique which has the capability to represent very small to very large functions inform of reversible circuit. As the ESOP-based designs contain very high number of gates, appropriate fault detection mechanism has to be employed over them to check their logical correctness. In the arena of testing, there are two ways to check the correctness of any circuit; (1) checking the circuit in offline mode or (2) in online mode. In compare to offline mode, online checking seems very efficient for testing as it does not take out the circuit from its operational mode for checking and find faults when the circuits are operational.

So, developing efficient algorithms for online testing always bear high importance and in this work we have developed a testing mechanism that not only easily finds any type of faults (*SMGF/PMGF/MMGF*/Cross-point) in reversible circuits but can deal with very large benchmarks as well. Our developed testing model has three phases. In the first phase, we build the testable design where in second phase we state about the fault detection mechanism. In the final phase, first we generate fault-specific test vectors and then employ the testing scheme.

Phase 1: Formation of Universal Testable Design

For easy of understanding, the way of transforming a circuit into its equivalent testable design is explained here using an example.

Let us consider the circuit (C_{test}) represented in Fig. 2a is to be converted into a testable form ($C_{testable}$) but the most important consideration in forming testable circuit is to reconfigure the circuit in such a manner that the main functionality of the circuit is not altered. In our approach, by maintaining that constraint, we have formed a testable design by appending an additional ancillary (*ANC*) line along with redundant gates into that circuit and the stated transformation has been conducted in the following three stages.

Step 1: In this step, we add one ancillary line (*ANC*) at the end of the circuit (C_{test}) inputs (see Fig. 2c).

Step 2: Here, all the additional circuit lines are connected to the circuit C_{test} such that for each individual gates—G_i (available in the input circuit C_{test}) there exist an identical gate—G_{ci} (*copy constituent gate*) whose control nodes are located at the same position as of *gate*—G_i.

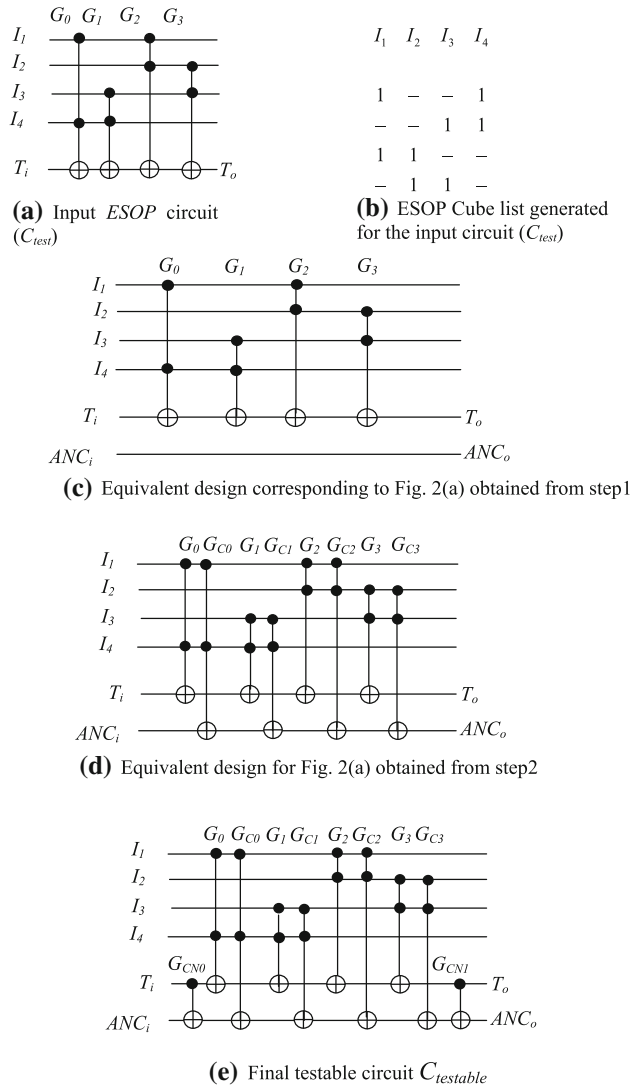
After implementation of the aforementioned steps, the circuit of Fig. 2c is converted to the design as depicted Fig. 2d.

Step 3: Now, two additional *CNOT* gates G_{CN0} and G_{CN1} are augmented to the circuit obtained in step2 whereby one of these additional gates is inserted before the first gate of the circuit and the other *CNOT* gate is added at the end of the circuit. In our case, we interpret such additional pairs as "*target line supporting gate.*"

Here after, the design as shown in Fig. 2d is transformed to its equivalent testable design ($C_{testable}$) as depicted in Fig. 2e.

Inspite of converting an input circuit to a testable design by appending redundant gates, the functionality of the circuit remains unaltered and this can be proved by finding logical expression obtained from the outputs of both the circuits—input and testable. Here, we are establishing that fact.

Let us consider the two circuits (input and testable circuit) as shown in Fig. 2a, e.



(a) Input *ESOP* circuit (C_{test})

(b) ESOP Cube list generated for the input circuit (C_{test})

(c) Equivalent design corresponding to Fig. 2(a) obtained from step1

(d) Equivalent design for Fig. 2(a) obtained from step2

(e) Final testable circuit $C_{testable}$

Fig. 2 **a** Input *ESOP* circuit (C_{test}), **b** ESOP cube list generated for the input circuit (C_{test}), **c** equivalent design corresponding to Fig. 2a obtained from step1, **d** equivalent design for Fig. 2a obtained from step 2, **e** final testable circuit $C_{testable}$

The logical expression at the output port (T_o) of input circuit depicted in Fig. 2a is $T_X = T_i \oplus I_1I_4 \oplus I_3I_4 \oplus I_1I_2 \oplus I_2I_3$. Again, find the value of output line (T_o) of Fig. 2e and it is $T_y = T_i \oplus I_1I_4 \oplus I_3I_4 \oplus I_1I_2 \oplus I_2I_3$. Now, from the evaluated expressions, it can be seen that both the outputs contain the same expression as $T_x \oplus T_y = 0$. Hence, it can be concluded that the testable design never changes the functional behavior of input circuit.

Phase 2: Fault Detection Mechanism

Till now, we have formed the testable design and thereby we state how to conclude that whether the input circuit contains faults or not. Depending on the applied test

vectors and the values at the two ports ANC_i and ANC_o can confirm this test.

Let us assume the circuit of Fig. 2e does not have any fault. Now, we collect the output from the port ANC_o and find that ANC_o=(ANC_i⊕ T_i)⊕ (I_1I_4⊕ I_3I_4⊕ I_1I_2⊕ I_2I_3)-(I_1I_4⊕ I_3I_4⊕ I_1I_2⊕ I_2I_3)⊕ T_i= ANC_i. This very nature provides an indication that if there are no faults in the circuit then the port value (ANC_o) at the ancillary line (*ANC*) will be same as the input port value (ANC_i) of the same line (*ANC*).

How this property helps to find faults in a circuit will be discussed in subsequent sections. But, the main parameter for detecting the faults is to find out appropriate test vectors, which is discussed next.

Phase 3: Applied Testing Schemes and Generation of Fault-Specific Test Vectors

In this phase, we test an input circuit for all possible faults. For each of the faults, first we have shown the type of test vectors required and then show how that fault-specific test vectors work to find and locate the erroneous junction.

For better understanding, the test procedure has been explained with an example and considering the same circuit of Fig. 2e to find out all type of stated faults.

Testing for *SMGF*

Phase 1: Test Vectors Generation

A circuit may poses an *SMGF* fault if either the *elementary gates* go missing from the circuit or *copy constituent gates* disappear from the test circuit. Accordingly, the test vectors also change once the fault type changes.

In the test vector generation process, the applied test vectors are obtained in a way to determine whether the disappearance of *elementary gates* or *copy constituent gates* is identical or not and such pattern strings can be retrieved from the ESOP *cube list* directly. Here also, in our example, the cube list of the input *ESOP* circuit (Fig. 2a) is used to generate the test vectors. Now, let consider the gate G_0 or G_{C0} disappears from the test circuit as given in Fig. 2e. So, we read the cubelist of the circuit and the pattern vector <1 - - 1> (see Fig. 2b) corresponding to the gate G_0 turns to be a test vector, where the symbol (-) represents do not care situation which can be filled with either 1 or 0. Hence from this pattern, the following four test vectors {<1001>, <101 1>, <1101>, <1111>} may be generated and can be applied at the input <I_1, I_2, I_3, I_4> of the test circuit to detect the fault.

As per the circuit drawn in Fig. 2e, for missing of gate G_1 or G_{C1}, the test vector is <- -11> (where 0, 1 ∈ -). Similarly, for missing of gate G_2 or G_{C2}, the test vector is <11 - -> (where 0, 1∈ -) and for missing of gate G_3 or G_{C3} the test vector will be <-11-> (where 0, 1 ∈ -).

Fault Detection

Now consider the gate G_0 misses from the test circuit ($C_{testable}$) of Fig. 2e and the same can be computed by collecting the values from port ANC_o. If the expression at the port ANC_o becomes reverse to the one located at port ANC_i, then definitely an error should originate in the testable circuit.

Now, we discuss the various possible cases when a specific gate disappears.

Case 1: Missing of elementary gates from $C_{testable}$

When gate G_0 in Fig. 2e disappears from $C_{testable}$ and makes it a faulty one, the expression from ANC_o of anicallary line *ANC* in $C_{testable}$ becomes (ANC_i⊕ T_i)⊕ (I_1I_4-I_3I_4⊕ I_1I_2⊕ I_2I_3)⊕ (I_3I_4⊕ I_1I_2⊕ I_2I_3)⊕ T_i= ANC_i⊕ I_1I_4.

Now, apply test vector <I_1,I_2,I_3,I_4> = <1- -1> (where 0, 1∈ -) over the circuit $C_{testable}$. So, ANC_o= ANC_i⊕1.1 =$\overline{ANC_i}$

As we find that the port values at ANC_o and ANC_i are opposite in polarity, so there is a fault in the design of Fig. 2e.

Case 2: Copy constituent gates in $C_{testable}$ go missing:

Let consider the gate G_{C0} goes missing from $C_{testable}$ (see in Fig. 2e). Missing of such gate G_{C0} makes the expression ANC_o as (ANC_i⊕ T_i)⊕ (I_3I_4⊕ I_1I_2⊕ I_2I_3)⊕ (I_1I_4⊕ I_3I_4⊕ I_1I_2⊕ I_2I_3)⊕ T_i= ANC_i⊕ I_1I_4 and after executing the test vector <1- -1> (where 0, 1 ∈ -) over $C_{testable}$, it is observed that the value assigned at line ANC_o has been toggled, i.e., ANC_o=-ANC_i⊕ I_1I_4=$\overline{ANC_i}$. From this, it can be confirmed that a gate is missing from $C_{testable}$.

Similarly, disappearance of other *copy constituent gates* such as G_{C1}, G_{C2}, G_{C3} from the circuit $C_{testable}$ in Fig. 2e can be detected. But, here an interesting observation can be found that the vectors used for identifying missing *copy constituent gates* are identical to the one used for detecting missing of *elementary gates.*

Testing for PMGF

Test Vector Generation for PMGF

Here, we are discussing about type of test vectors required for testing of *PMGF* type fault. In the due process, while generating the test vectors, first we read the cubes from the cubelist and then complement each bit (0 or 1) of every cube. The cubes with the complemented bit positions are

Fig. 3 Test vector generation to detect *PMGF* in **a** G_0 of Fig. 2e, ▶ **b** G_1 of Fig. 2e, **c** G_2 of Fig. 2e, **d** G_3 of Fig. 2e

the test vectors to detect all *PMGFs* for that gate whose input cube was considered.

For an ease of understanding, let us discuss the process with the circuit of Fig. 2e in which we will inject PMGF and will detect it using our generated test vectors. But in Fig. 2e, *PMGFs* can appear either in *elementary gates* or in *copy constituent gates* positions, so, we are considering both the conditions separately.

From the Fig. 2(b), we can see that the first cube <1–1> of the cube list describes the gate details of G_0 that the gate G_0 has two controls at positions I_1 and I_4. Now, by flipping the control bit 1 and by ensuring the presence of at least one positive control in the test pattern, two test vectors are generated—{<0–1> , {<1–0>} (see Fig. 3a).

Similarly, the test vectors for rest of the gates G_1, G_2 and G_3 also can be found in the same way (the generated test vectors for different gates are shown in Fig. 3b–d, respectively). The test vectors generated for *elementary gates* are also used for *PMGF* detection in *copy constituent gates* as the applied test vectors to detect both the faults are same.

PMGF Detection Phase

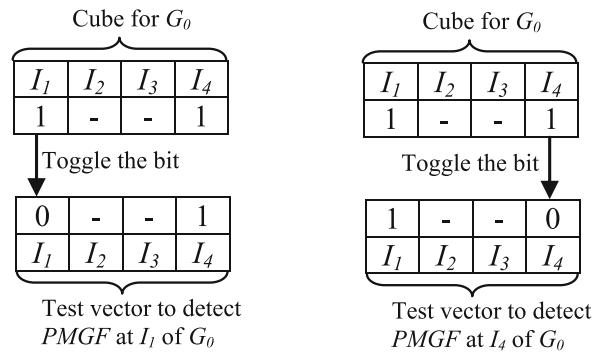
As like *SMGF* detection process, *PMGF* also can be detected by judging the input/output expressions at ancillary line (*ANC*). Similar to SMGF, PMGF may originate in *elementary gates* and *copy constituent* gates. Let us consider both the scenarios.

Case 1: Control node goes missing from *elementary gates* of the testable design:

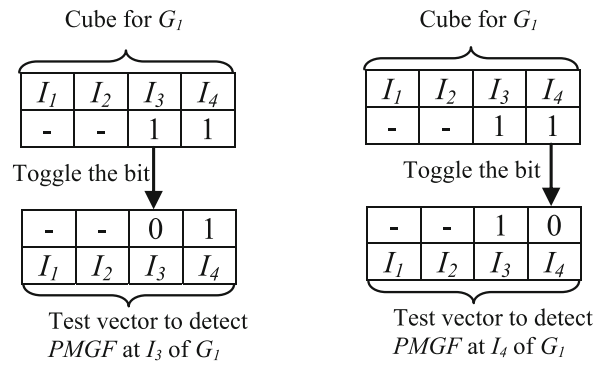
Let us assume that the control of gate G_0 at line I_1 is missing and it has caused a *PMGF* in the design of Fig. 2e. Now, we find the value at $ANC_o=(ANC_i\oplus T_i)\oplus (I_1I_4\oplus I_3I_4\oplus I_1I_2\oplus I_2I_3)\oplus (I_4\oplus I_1I_2\oplus I_2I_3)\oplus T_i= ANC_i\oplus I_1I_4\oplus I_4= ANC_i\oplus I_4(I_1\oplus 1)= ANC_i\oplus \overline{I_1}\, I_4$. Again, when we apply the test vector <0 - -1>, the expression of ANC_o becomes $ANC_i\oplus \bar{0}.1 =\overline{ANC_i}$. So, as both the port values are opposite to each other, this very fact confirms the presence of the fault.

Now, if we assume that inspite of control node at line I_1, the node misses from the same gate at line I_4. In this situation, first we compute the port value $ANC_o= ANC_i\oplus I_1\,\overline{I_4}$ and then the presence of the fault is further confirmed when the test vector <1 - -0> is applied over that which yield $ANC_o= ANC_i\oplus 1.\bar{0} =\overline{ANC_i}$.

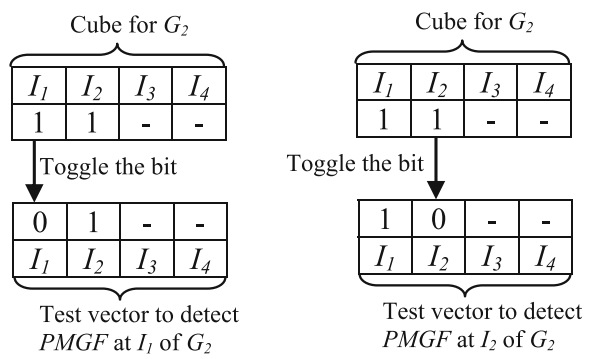
Similarly, missing of control nodes from remaining *elementary* gates also can be found by following the same strategy.



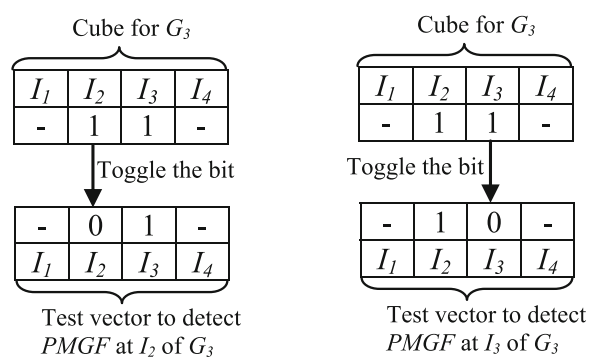
(a) Test vector generation to detect *PMGF* in G_0 of Fig. 2(e).



(b) Test vector generation to detect *PMGF* in G_1 of Fig. 2(e).



(c) Test vector generation to detect *PMGF* in G_2 of Fig. 2(e).



(d) Test vector generation to detect *PMGF* in G_3 of Fig. 2(e).

Case 2: Control is missing from *copy constituent gate* of the testable design:

Let us assume a control of gate G_{C0} at line I_1 goes missing from $C_{testable}$ in Fig. 2e.

Due to disappearance of the said control point of gate G_{C0}, the output expression ANC_o becomes, ANC_o=($ANC_{i\text{-}o}$=($ANC_i \oplus T_i$)$\oplus$ ($I_4 \oplus I_3I_4 \oplus I_1I_2 \oplus I_2I_3$)$\oplus$ ($I_1I_4 \oplus I_3I_4 \oplus I_1I_2 \oplus I_2I_3$)$\oplus$ T_i= $ANC_i \oplus I_4 \oplus I_1I_4$ = $ANC_i \oplus I_4(I_1 \oplus 1)$= $ANC_{i\text{-}}\overline{I_1}\, I_4$.

As the applied test vectors to detect missing of controls from *copy constituent gates* and *elementary gates* are same, hence, we apply the test vector <0 - -1> over the generated expression and it confirms the presence of fault as it produce ANC_o= $ANC_i \oplus \bar{0}\ 1$=$\overline{ANC_i}$.

Like as the previous scenario, disappearance of other control nodes from *copy constituent gates* like G_{C1}, G_{C2}, G_{C3} in circuit $C_{testable}$ of Fig. 2e also can be detected in the same way.

Testing for Control Appearance Fault

Test Vector Generation for Cross-Point Fault

Like as *PMGF*, control appearance faults may originate in both type of gates *elementary* and *copy constituent*.

In control appearance fault, while generating the test vectors, initially we read the individual cubes from the list and replace the dash (-) positions with '0' bit value. We have seen that if an *n*-variable cube contains *m* (where $m < n$) number of dash (-) positions, then the total number test vectors will be *m* to find a single node appearance fault.

For the design of Fig. 2e, if we assume that single control node may appears in either of the positions in the first gate (G_0) then to detect the fault, the corresponding test vectors will be {<10-1 >, <1-01>} (shown in Fig. 4a).

Similarly, the test vectors for possible control appearance fault in gate G_1, G_2 and G_3 of Fig. 2e also can be computed (see in Fig. 4b–d).

Here, we are not discussing the way for tracking control appearance fault in *copy constituent gates* as the test vectors for *elementary gates* are same as of *copy constituent gates*.

Appearance Fault Detection Phase

The appearance fault in testable design can also be detected by following the same strategy adhered in SMGF and PMGF detection.

Now, let us consider the different situations when faults may originate.

Case 1: When a control appears in *elementary gates* of the testable design:

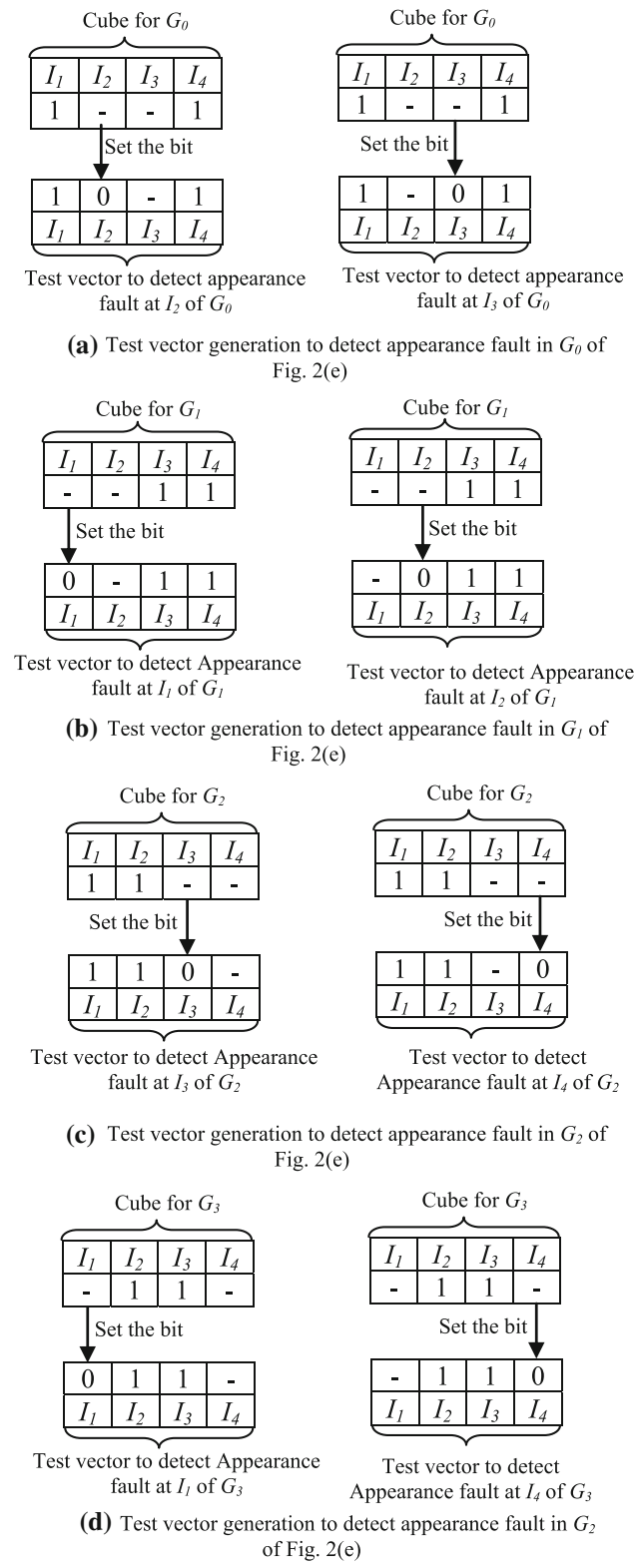


Fig. 4 Test vector generation to detect appearance fault in **a** G_0 of Fig. 2e, **b** G_1 of Fig. 2e, **c** G_2 of Fig. 2e, **d** G_2 of Fig. 2e

Let us assume that a control has appeared in line I_2 of gate G_0 and this makes the design of Fig. 2e faulty.

Now, compute the output value at line *ANC*: ANC_o=($-ANC_i \oplus T_i$)$\oplus$ ($I_1I_4 \oplus I_3I_4 \oplus I_1I_2 \oplus I_2I_3$)$\oplus$ ($I_1I_2I_4 \oplus I_1I_2 \oplus I_2I_3$)$\oplus$ $T_i = ANC_i \oplus I_1I_2I_4 \oplus I_1I_4 = ANC_i \oplus I_1I_4(I_2 \oplus 1) = ANC_i \oplus I_1 \overline{I_2} I_4$.

Again, if we apply the test vector <1 0 - 1> then we find that $ANC_o = ANC_i \oplus 1\ \bar{0}.1 = \overline{ANC_i}$ and as both the values are inverted, it confirms that there is a fault in the design of Fig. 2e.

If the control appears in gate G_0 at line I_3, then also in the same way fault can be detected and again the presence of fault is confirmed by applying test vector <1 - 0 1> over the circuit which yield $ANC_o = \overline{ANC_i}$.

The same process can be followed in detecting control appearance faults for other gates as well.

Now, consider a different situation when control appears in *copy constituent gates* of the testable design. Like as the process of detecting *SMGFs/PMGFs* in *copy constituent gates*, control appearance fault in *copy constituent gates* of the testable design ($C_{testable}$) can be detected by applying the same set of test vectors applied while detecting appearance fault in *elementary gates*.

Testing for MMGF

As like other testing techniques, this testing scheme also has two phases—*test vector generation* and *fault detection phase*.

Test Vector Generation for MMGF

Here, we define the test vector for a set of gates G_0, G_1,… G_i, as the logical OR in between the cubes of the respective selected gates and now we will see how this vector become very sufficient to find all the multiple missing-gate faults in a circuit.

For an ease of understanding, let us consider the circuit of Fig. 2e and assume that gates G_0, G_1 and G_2 are missing from the testable design. Now, we have to generate the appropriate test vectors which can detect any partial missing-gate faults for the selected gates.

So, as we have said earlier we perform bitwise OR between the corresponding cubes (<1 - - 1>, <- - 1 1> and <1 1 - - >) for gates G_0, G_1 and G_2 and the new test vector is:

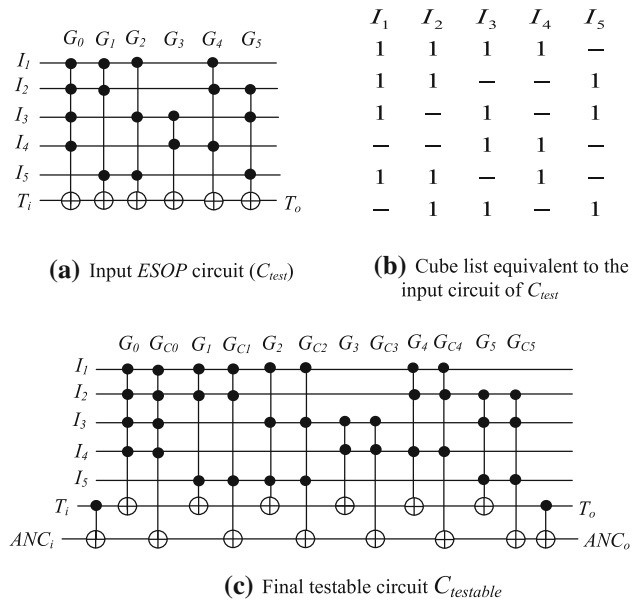
$$<1--1> \wedge <--11> \wedge <--11>$$

(where symbol signifies OR operation)

$$= <1111>$$

MMGF Fault Detection Phase

Here, we apply the previously selected test vector over the circuit and check the output values over the line *ANC*.



(a) Input *ESOP* circuit (C_{test})

(b) Cube list equivalent to the input circuit of C_{test}

(c) Final testable circuit $C_{testable}$

Fig. 5 **a** Input *ESOP* circuit (C_{test}), **b** cube list equivalent to the input circuit of C_{test}, **c** final testable circuit $C_{testable}$

Let assume, gates G_0, G_1 and G_2 disappear from the design of Fig. 2(e). Now, it can be seen from the testable design $C_{testable}$, that ANC_o=($ANC_i \oplus T_i$)$\oplus$ ($I_1I_4 - I_3I_4 \oplus I_1I_2 \oplus I_2I_3$)$\oplus$ (I_2I_3)$\oplus$ $T_i = ANC_i \oplus I_1I_4 \oplus I_3I_4 \oplus I_1I_2$ and when the test vector <1111> is applied over it, we find that $ANC_o = ANC_i - 1.1 \oplus 1.1 \oplus 1.1 = \overline{ANC_i}$ which implies that the circuit is faulty.

Still so far we have discussed over various types of faults and their respective testing schemes, and now to make this understanding clear, here, we have given one more example.

Example 1 Let us consider the input circuit of Fig. 5a and its corresponding *cubelist* in Fig. 5b.

The testable design of the input circuit is first obtained in Fig. 5c. Before injecting any fault in the design, we check the circuit correctness and find that ANC_o=($ANC_i - T_i$)$\oplus$ ($I_1I_2I_3I_4 \oplus I_1I_2 \quad I_5 \oplus I_1I_3I_5 \oplus I_3I_4 \oplus I_1I_2I_4 \oplus I_2I_3I_5$)$-_5 \oplus I_1I_3I_5 \oplus I_3I_4 \oplus I_1I_2I_4 \oplus I_2I_3I_5$)$\oplus$ ($I_1I_2I_3I_4 \oplus I_1I_2 \quad I_5 \oplus I_1I_3 -_5 \oplus I_1I_3I_5 \oplus I_3I_4 \oplus I_1I_2I_4 \oplus I_2I_3I_5$)$\oplus$ $T_i = ANC_i$. As both the values ANC_o and ANC_i are same, so we conclude that the circuit is a fault-free design.

Now, here, we are not briefing all the different type fault models in detail over the circuit again and only we are mentioning the respective fault category and their applied test vectors.

The test vectors for *SMGFs* are <1111->, <11–1> , <1-1-1>, <–11->, <11-1->, <-11-1>.

Table 1 Comparison of different online testable approaches

Benchmark name	Approach in [28]		Approach in [24]		Approach in [27]		Approach in [23]		Our approach	
	GO	QC	GO	QC	GO	QC	GO	QC	GO	QC
f51m	14	76,126	29,982	128,927	2670	164,889	29,982	128,927	14	74,802
frg2	143	32,384	50,722	218,109	4048	247,332	50,722	218109	143	24,938
sqrt8	8	1324	1322	5689	122	7086	1322	5689	8	1246
sym9	9	14,140	5732	24,652	528	31,890	5732	24,652	9	13,884
rd73	7	2446	3832	16,482	342	20,583	3832	16,482	7	2288
d17	10	3216	2412	10,376	186	11,478	2412	10,376	10	3120
misex3	14	401,858	32,872	141,354	2816	174,732	32,872	141,354	14	398,356
apla	10	7036	4142	17,815	334	20,574	4142	17,815	10	6878
table3	14	162,102	51,872	223,054	4294	265,566	51,872	223,054	14	160,080
z4ml	6	1380	902	3883	88	5070	902	3883	6	1286
max46	9	11,102	3562	15,321	326	19,314	3562	15,321	9	10,890
cu	14	2376	1292	5560	110	6234	1292	5560	14	2298
bw	5	2500	4422	19,019	386	25,140	4422	19,019	5	1888
sqn	7	4396	2612	11,236	240	14,256	2612	11,236	7	4246
cm82a	5	354	502	2163	50	2823	502	2163	5	310
con1	7	456	382	1647	38	1929	382	1647	7	414
ex2	5	312	402	1733	42	2241	402	1733	5	284
ex5p	8	4980	19,732	84,852	1540	98,898	19,732	84,852	8	3688
9symml	9	28,644	5952	25,598	532	32,220	5952	25,598	9	28,388
apex5	117	26,516	45,362	195,061	3518	213,843	45,362	195,061	117	20,700
Average	21.1	39,182.4	13,400.5	57,626.55	1110.5	68,304.5	13,400.5	57,626.55	21.1	37,999.2

Again consider the same testable design of Fig. 5(c) and assume gate G_0 is missing from the circuit then the output expression ANC_o becomes:

ANC_o=(ANC_i⊕ T_i)⊕ ($I_1I_2I_5$⊕ $I_1I_3I_5$⊕ I_3I_4⊕ $I_1I_2I_4$⊕ $I_2I_3I_5$)⊕ ($I_1I_2I_3I_4$⊕ I_1I_2 I_5⊕ $I_1I_3I_5$⊕ I_3I_4⊕ $I_1I_2I_4$⊕ $I_2I_3I_5$)⊕ T_i= ANC_i⊕ $I_1I_2I_3I_5$⊕ $I_1I_3I_5$⊕ I_3I_4⊕ $I_1I_2I_4$⊕ $I_2I_3I_5$)⊕ T_i= ANC_i⊕ $I_1I_2I_3I_4$.

Now, if we apply the test vector <1111-> over the circuit then ANC_o would be $\overline{ANC_i}$. Hence, SMGF is detected.

Similarly, test vectors for the detection of *PMGFs* are <0111->, <1011->, <1101->, <1110->, <01–1>, <10–1>, <11–0>, <0-1-1>, <1-0-1>, <1-1-0>, <–01->, <–10->, <01-1->, <10-1->, <11-0->, <-01-1>, <-10-1>, <-11-0>.

If we assume that the control of gate G_0 is missing from the Fig. 5c then ANC_o=(ANC_i⊕ T_i)⊕ ($I_2I_3I_4$⊕ I_1I_2 I_5⊕ I_1I_5⊕ $I_1I_3I_5$⊕ I_3I_4⊕ $I_1I_2I_4$⊕ $I_2I_3I_5$)⊕ ($I_1I_2I_3I_4$⊕ I_1I_2 I_5⊕ $I_1I_3I_5$⊕ $I_1I_3I_5$⊕ I_3I_4⊕ $I_1I_2I_4$⊕ $I_2I_3I_5$)⊕ T_i= ANC_i⊕ $\overline{I_1}$ $I_2I_3I_4$ and the test vector <0111-> can detect the *PMGF*.

Different test vectors for control appearance faults are <11110>, <110-1>, <11-01>, <101-1>, <1-101>, <0-11->, <-011->, <–110>, <1101->, <11-10>, <011-1>, <-1101>.

Again, if we assume that there is a control appearance fault in the design of Fig. 5c then ANC_o=(ANC_i⊕ T_i)⊕ ($I_1I_2I_3I_4$ I_5⊕ I_1I_2 I_5⊕ $I_1I_3I_5$⊕ I_3I_4⊕ $I_1I_3I_5$⊕ I_3I_4⊕ $I_1I_2I_4$⊕ $I_2I_3I_5$)⊕ ($I_1I_2I_3I_4$⊕ I_1I_2 I_5⊕ $I_1I_3I_5$⊕ I_3I_4⊕ $I_1I_2I_4$⊕ $I_2I_3I_5$)⊕ T_i= ANC_i⊕ $I_1I_2I_3I_4$ $\overline{I_5}$ and the test vector <11110> can detect the fault as it finds ANC_o = $\overline{ANC_i}$.

Now, again if we consider that gates G_0, G_1 and G_2 are missing from the circuit of Fig. 5c and a MMGF fault has originated in the circuit then the following test vectors {<1111-> ∧ <11- -1> ∧ <1-1-1> = <11111>} become very sufficient to find the said fault.

Again, the presence of fault is further confirmed as ANC_o=(ANC_i⊕ T_i)⊕ (I_3I_4⊕ $I_1I_2I_4$⊕ $I_2I_3I_5$)⊕ ($I_1I_2I_3I_4$⊕ I_1I_2 I_5⊕ $I_1I_3I_5$⊕ I_3I_4⊕ $I_1I_2I_4$⊕ $I_2I_3I_5$)⊕ T_i= ANC_i⊕ $I_1I_2I_3I_4$⊕ $I_1I_3I_5$⊕ I_3I_4⊕ $I_1I_2I_4$⊕ $I_2I_3I_5$)⊕ T_i= ANC_i⊕ $I_1I_2I_3I_4$ = ANC_i⊕ I_3I_4⊕ $I_1I_2I_4$⊕ $I_2I_3I_5$ and the applied test vector <11111> finds ANC_o = $\overline{ANC_i}$. Hence, *MMGF* is detected.

Similarly, other type of faults also can be detected by following the same strategy mentioned previously.

Experimental Results

We have tested our technique over a wide range of benchmarks [29] and obtained results are summarized in Tables 1, 2, 3 and 4. In Table 1, the computed results have

Table 2 Improvements achieved by our approach

	Parameters	Existing approaches			
		Approach in [28]	Approach in [24]	Approach in [27]	Approach in [23]
Achieved improvements	GO	0%	99.84%	98.10	99.84%
	QC	3%	34.05%	44.03%	34.05%

been compared with the works from [23, 24, 27] and [28], where two cost parameters quantum cost (QC) and garbage output (GO) have been evaluated. Improvements over existing works are summarized in Tables 2 and 3. From Table 2, it can be seen that our developed scheme has attained 34%, 44% and 3% improvements when QC is considered and an improvement of 99%, 98% is achieved when the comparison is made with respect to GO. The design overhead of testable circuits is computed in Table 3. Comparison with a recent online testing work [30] is presented in Table 4, where considerable improvement has been achieved in our testing scheme.

Table 3 Overhead calculation of the testable design over the non-testable design

Approach	Average overhead for testability feature	
	GO	QC
Approach in [28]	0%	300.67%
Approach in [24]	63,409.48%	312.28%
Approach in [27]	5163.03%	388.67%
Approach in [23]	63,409.48%	312.28%
Our approach	0%	100%

Limitation of the work: To be specific, our proposed online testing algorithm is for a special class of reversible circuit—ESOP-based designs and this testing scheme cannot be employed over all type of circuits. Another issue to be mentioned that though we are successful in detection of different type of faults including appearance fault, but this scheme cannot deal with those circuits having negative control MCT gates in their designs.

Conclusion

In this work, we have shown an efficient online testing scheme for detecting all type of faults in ESOP-based designs with minimum number of test vectors. Not only the traditional faults can be tracked using our technique, but rare type of faults like appearance fault also can be traced by employing our testing algorithm. The developed scheme has been tested over different benchmark functions, and it has been seen that the presented technique finds faults in circuits very effectively and can deal with very large benchmarks as well. Comparison with existing testing approaches also has been made and considerable improvement is registered.

Table 4 Comparison with online work [30]

Benchmark name	GC	Approach in [30]		Our Approach		Our Approach	
		GC	Design overhead (%)	GC	Design overhead (%)	Reduced GC	Reduced design overhead (%)
apex5	2909	7868	171	5820	102	2048	49
Apla	80	180	125	162	102	18	23
Cm82a	22	54	145	46	102	8	43
Cu	40	108	166	82	102	26	64
Dk17	49	118	137	100	102	18	35
Ex2	15	40	166	32	102	8	64
Frg2	3724	7594	110	7450	102	44	8
Sqrt8	40	96	140	82	102	14	38
Sym9	28	74	164	58	102	16	62
Z4ml	48	110	130	98	102	12	28

References

1. R. Landauer, Irreversibility and heat generation in the computing process. IBM J. Res. Dev. **5**, 183–191 (1961)
2. C.H. Bennett, Logical reversibility of computation. IBM J. Res. Dev. **17**, 525–532 (1973)
3. A. Berut, A. Arakelyan, A. Petrosyan, S. Ciliberto, R. Dillenschneider, E. Lutz, Experimental verification of Landauer's principle linking information and thermodynamics. Nature **483**(3), 187–189 (2012)
4. M. Nielsen, I. Chuang, *Quantum Computation and Quantum Information* (Cambridge University Press, Cambridge, 2000)
5. R. Wille, O. Keszocze, S. Hillmich, M. Walter, A.G. Ortiz, Synthesis of approximate coders for on-chip interconnects using reversible logic, in *Design*, *Automation and Test in Europe* (2016)
6. A. Rauchenecker, T. Ostermann, R. Wille, Exploiting reversible logic design for implementing adiabatic circuits, in *International Conference on Mixed Design of Integrated Circuits and Systems* (2017)
7. K. Ramasamy, R. Tagare, E. Perkins, M. Perkowski, Fault localization in reversible circuits is easier than for classical circuits, in *Proceedings of International Workshop on Logic and Synthesis* (2004)
8. A. Chakraborty, Synthesis of reversible circuits for testing with universal test set and C-testability of reversible iterative logic arrays, in *Proceedings of VLSI Design* (2005), pp. 249–254
9. H. Rahaman, D.K. Kole, D.K. Das, B.B. Bhattacharya, Fault diagnosis for missing-gate fault (SMGF) model in reversible quantum circuits. Int. J. Comput. Electr. Eng. **37**, 475–485 (2011)
10. B. Mondal, D.K. Kole, H. Rahaman, D.K. Das, Generator for Test Set Construction of SMGF in Reversible Circuit by Boolean difference method, in *Proceedings of Asian Test Symposium* (2014), pp. 68–73
11. B. Mondal, C. Bandyopadhyay, D. K. Kole, J. Mathew, H. Rahaman, Diagnosis of SMGF in ESOP based reversible logic circuit, in *International Symposium on Electronic System Design* (2014), pp. 89–93
12. J. Mondal, D.K. Das, D.K. Kole, H. Rahaman, B.B. Bhattacharya, On designing testable reversible circuits using gate duplication, in *International Symposium on VLSI Design and Test* (2013), pp. 322–329
13. D.K. Kole, H. Rahaman, D.K. Das, B.B. Bhattacharya, Derivation of optimal test set for detection of multiple missing-gate faults in reversible circuits, in *Proceedings of Asian Test Symposium* (2010), pp. 33–38
14. S.N. Mahammad, S.K.S. Hari, S. Shroff, V. Kamakoti, Constructing online testable circuits using reversible logic, in *International Symposium on VLSI Design and Test* (2006), pp. 373–383
15. T. Toffoli, Reversible Computing, *Technical Memo MIT/LCS/TM-151*, MIT Lab for Computer Science (1980)
16. E. Fredkin, T. Toffoli, Conservative logic. Int. J. Theor. Phys. **21**(3–4), 219–253 (1982)
17. R.P. Feynman, *Feynman Lectures on Computation* (Perseus Books, New York, 1996)
18. K. Fazel, M.A. Thornton, J.E. Rice, ESOP-based Toffoli gate cascade generation, in *Pacific Rim Conference on Communications, Computers and Signal Processing (PacRim)*, Victoria, Canada (2007), pp. 206–209
19. J.P. Hayes, I. Polian, B. Becker, Testing for missing-gate faults in reversible circuits, in *Proceedings of Asian Test Symposium* (2004), pp. 100–105
20. M. Perkowski, J. Biamonte, M. Lukac, Test generation and fault localization for quantum circuits, in *Proceedings of International Symposium on Multi-Valued Logic* (2005), pp. 62–68
21. I. Polian, J.P. Hayes, T. Fienn, B. Becker, A family of logical fault models for reversible circuits, in *Proceedings of Asian Test Symposium* (2005), pp. 422–427
22. J. Zhong, J.C. Muzio, Analyzing fault models for reversible logic circuits, in IEEE *congress on evolutionary conputation* (2006), pp. 219–253
23. D.P. Vasudevan, P.K. Lala, J.P. Parkerson, Online testable reversible logic circuit design using NAND blocks, in *International Symposium on Defect and Fault-Tolerance in VLSI Systems* (2004), pp. 324–331
24. D.P. Vasudevan, P.K. Lala, D. Jia, J.P. Parkerson, Reversible logic design with online testability. IEEE Trans. Instrum. Meas. **55**, 406–414 (2006)
25. S.N. Mahammad, S. Hari, S. Shroff, V. Kamakoti, Constructing online testable circuits using reversible logic, in *International Symposium on VLSI Design and Test* (2006), pp. 373–383
26. S.N. Mahammad, K. Veezhinathan, Constructing online testable circuits using reversible logic. IEEE Trans. Instrum. Meas. **59**, 101–109 (2010)
27. N. Farazmand, M. Zamani, M.B. Tahoori, Online fault testing of reversible logic using dual rail coding, in *Proceedings of International On-Line Testing Symposium* (2010), pp. 204–205
28. D.K. Kole, H. Rahaman, D.K. Das, Synthesis of online testable reversible circuit, in *International Symposium on Design and Diagnostic of Electronic Circuits and Systems* (2010), pp. 277–280
29. R. Wille, D. Grosse, L. Teuber, G.W. Dueck, R. Drechsler, Revlib: an online resources for reversible functions and reversible circuits. ISMVL **24**, 220–225 (2008)
30. M.A. Nashiry, G.G. Bhaskar, J.E. Rice, Online testing for three fault models in reversible circuits, in *ISMVL* (2015)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.